

DOCUMENT MANAGEMENT SYSTEM

React/Vite to Java JSP/Servlet PWA

Migration Specification and Implementation Instructions



Prepared as a handover brief for modifying the existing project code

Recommended frontend	JSP + HTML + CSS + Vanilla JavaScript
Backend pattern	Java Servlets + JavaBeans + DAO + JDBC
Database	Oracle Database 21c
Deployment	Apache Tomcat 9
Build tool	Maven

How to Use This Document

Send this PDF together with the current project ZIP files to the chat or developer responsible for modifying the application. The recipient should inspect the existing code first, preserve useful screens and business rules, and then perform the migration according to this specification.

Primary instruction

Do not rebuild the application blindly. Review the existing React/Vite and backend code, identify reusable UI, assets, validation rules and database models, then convert the project incrementally into the target Java MVC PWA architecture.

Contents

Section	Topic
1	Migration Objective
2	Target Technology Stack
3	System Architecture
4	Functional Scope to Preserve
5	React/Vite-to-JSP Migration Mapping
6	Required Maven Project Structure
7	Implementation and Migration Phases
8	PWA, Security and File-Handling Rules
9	Oracle Database Guidance
10	Acceptance Criteria
11	Ready-to-Paste Instruction for Another Chat

1. Migration Objective

The current project uses React and Vite for its frontend. The objective is to convert it into a mobile-first, installable Progressive Web Application that uses technologies already familiar to the project owner: Java JSP, Java Servlets, JavaBeans, DAO, JDBC, HTML, CSS and vanilla JavaScript.

The final application should be deployable as one Maven WAR package on Apache Tomcat 9 and should connect to Oracle Database 21c. It should not require React, Vite, JSX, TypeScript, Node.js or a separate frontend build process during normal development and deployment.

Migration principles

- Preserve the existing visual design, project colour scheme and working features wherever practical.

- Convert React components into reusable JSP fragments, JSP pages and vanilla JavaScript modules.
- Replace React Router navigation with Servlet URL mappings and JSP forwarding or redirects.
- Replace client-side application state with request, session and database-backed state as appropriate.
- Use Java Servlets as controllers, JavaBeans as models and DAO classes for Oracle data access.
- Keep the application responsive and optimized for mobile devices while remaining usable on desktop browsers.
- Add PWA installation support without caching confidential documents or authenticated pages.

2. Target Technology Stack

Layer	Technology	Responsibility
User interface	JSP, HTML5, CSS3 and vanilla JavaScript	Responsive pages, forms, dashboards and interactions
Controller	Java Servlets using javax.servlet	Request processing, navigation, validation and authorization
Model	JavaBeans / POJO classes	User, document, approval, department, notification and audit data
Data access	DAO + JDBC	Parameterized SQL and Oracle transactions
Database	Oracle Database 21c	Users, roles, documents, routing, approval history and metadata
PWA layer	manifest.json + service-worker.js	Installation, app shell caching and offline fallback
Server	Apache Tomcat 9	WAR deployment and HTTP session management
Build	Maven	Dependencies, compilation, testing and WAR packaging
Development	VS Code or NetBeans	Source editing, debugging and deployment

Tomcat 9 compatibility rule

Use the javax.servlet namespace and Servlet API 4.0. Do not mix Jakarta Servlet packages from Tomcat 10 or Spring Boot 3 into the Tomcat 9 project.

Recommended runtime baseline

- Java 17, or the Java version already used successfully in the environment.
- Apache Tomcat 9.x.
- Maven WAR packaging.
- Oracle JDBC driver suitable for the selected Java version, such as ojdbc11 for Java 11 or later.

3. System Architecture

```

Mobile / Desktop Browser
    |
    v
JSP + HTML + CSS + Vanilla JavaScript
    | HTTP requests / form submissions
    v
Java Servlets (Controllers)
    |
    v
Service classes and DAO classes
    | JDBC
    v
Oracle Database 21c

Static PWA layer: manifest.json + service-worker.js + offline.html
  
```

MVC responsibility

Area	Implementation	Rule
View	JSP pages and JSP fragments	Render data supplied by a servlet. Avoid direct SQL and large Java scriptlets inside JSP.
Controller	Servlet classes	Receive requests, validate inputs, check roles, call services/DAOs and forward or redirect.
Model	JavaBeans	Represent application data using private fields, constructors, getters and setters.
Data access	DAO classes	Execute prepared statements, map result sets and manage transactions.
Cross-cutting	Filters and utilities	Authentication, authorization, encoding, audit logging, database connection and upload validation.

4. Functional Scope to Preserve

The conversion must preserve the intended workflow and existing working functions. At minimum, the following roles and capabilities are required.

Role	Required capabilities
Clerk / System Administrator	Login; capture or upload documents; choose boss and destination; manage users and roles; view history; recall eligible documents; receive rejection or destination-change notices.

Role	Required capabilities
Boss / Approver	View pending items; inspect document metadata and attachments; approve or reject; approve individual or selected items; change destination before approval; record remarks.
IT / Finance / Management User	Login and view documents routed to the assigned department folder; search, filter and download authorized documents.

Required document workflow

1. The clerk captures an image using a phone camera or uploads an image, PDF, Word or Excel file.
2. The clerk enters document details, selects a boss, selects a destination folder and submits the item.
3. The document status becomes PENDING_APPROVAL.
4. The boss reviews the document and either approves, rejects or changes the destination before approval.
5. A rejected item returns to the clerk for correction and resubmission.
6. An approved item becomes available to users of the selected department folder.
7. Every submission, approval, rejection, destination change, recall and download should be recorded in an audit history.

Recommended statuses

DRAFT
 PENDING_APPROVAL
 REJECTED
 APPROVED
 ROUTED
 RECALLED
 ARCHIVED

5. React/Vite-to-JSP Migration Mapping

Current React/Vite concept	Target replacement	Migration note
React component (.jsx/.tsx)	JSP page, JSP fragment or reusable include	Preserve HTML structure and CSS classes; remove JSX syntax.
React Router	Servlet @WebServlet mappings and response redirects	Use context-path-safe URLs.
useState / Context state	Request attributes, session attributes, form fields or database records	Use session only for user/session data, not large documents.
useEffect data loading	Servlet doGet, DAO call and request forwarding	Load server data before rendering JSP.
fetch/Axios API calls	HTML form POST or vanilla fetch to a servlet	Use JSON only where asynchronous interaction is genuinely useful.

Current React/Vite concept	Target replacement	Migration note
Vite public assets	src/main/webapp/assets	Update all paths to include the application context path.
Vite environment variables	properties file, JNDI or environment variables	Never expose database credentials to the browser.
React conditional rendering	JSTL c:if, c:choose and EL	Avoid Java scriptlets in JSP.
React list rendering	JSTL c:forEach	Escape user-controlled output.
Client-side authentication token	Tomcat HttpSession and authentication filter	Use secure, HttpOnly session cookies.
npm build output	Maven WAR package	One deployable artifact for Tomcat 9.

Reuse guidance

Existing HTML, CSS, icons, images, form labels, validation messages and layout ideas may be reused. React-specific files, JSX syntax, TypeScript types, Vite configuration and npm-only runtime assumptions must be removed or translated.

6. Required Maven Project Structure

```

DocumentManagementSystem/
|-- pom.xml
|-- src/main/java/com/chekrol/dms/
|   |-- controller/
|   |   |-- LoginServlet.java
|   |   |-- LogoutServlet.java
|   |   |-- DocumentServlet.java
|   |   |-- DocumentUploadServlet.java
|   |   |-- ApprovalServlet.java
|   |   |-- UserManagementServlet.java
|   |-- model/
|   |   |-- User.java
|   |   |-- DocumentRecord.java
|   |   |-- Approval.java
|   |   |-- Department.java
|   |   |-- AuditLog.java
|   |-- dao/
|   |   |-- UserDAO.java
|   |   |-- DocumentDAO.java
|   |   |-- ApprovalDAO.java
|   |   |-- AuditLogDAO.java
|   |-- service/
|   |-- filter/
|   |   |-- AuthenticationFilter.java
|   |   |-- AuthorizationFilter.java
|   |-- util/
|   |   |-- DatabaseConnection.java
|   |   |-- FileStorageUtil.java
|   |   |-- ValidationUtil.java
|-- src/main/resources/
|   |-- database.properties
|-- src/main/webapp/
|   |-- index.jsp
|   |-- login.jsp

```

```

|-- WEB-INF/
|   |-- web.xml
|   |-- views/
|       |-- clerk/
|       |-- boss/
|       |-- department/
|       |-- common/
|-- assets/css/
|-- assets/js/
|-- assets/icons/
|-- manifest.json
|-- service-worker.js
|-- offline.html

```

Minimum Maven dependency categories

- javax.servlet-api 4.0.1 with provided scope.
- JSTL 1.2 for c:if, c:forEach and related tags.
- Oracle JDBC driver compatible with the selected Java version.
- A password hashing library such as jBCrypt.
- Optional connection pooling such as HikariCP, if appropriate for the deployment.
- Maven WAR Plugin configured for Tomcat deployment.

7. Implementation and Migration Phases

Phase	Expected work
Phase 1 - Inspect and inventory	Open both existing ZIP files. Identify frontend pages, components, routes, assets, validation, API calls, backend classes, database scripts and working features. Produce a brief inventory before deleting anything.
Phase 2 - Create Maven foundation	Create the WAR project, packages, database connection, base JSP layout, error pages, session authentication and role filters.
Phase 3 - Convert shared UI	Move styles and assets, convert header/navigation/cards/modals into JSP fragments and vanilla JavaScript, and apply the approved colour scheme.
Phase 4 - Convert authentication and users	Implement login, logout, session handling, password hashing, role checks and user management.
Phase 5 - Convert document workflow	Implement capture/upload, metadata, destination selection, boss selection, submission, approval, rejection, resubmission and folder access.
Phase 6 - Add history and notifications	Implement audit records, pending counts, overdue indicators and user notifications.
Phase 7 - Add PWA layer	Add manifest, icons, service worker, installability and an offline fallback without caching confidential content.
Phase 8 - Test and package	Test roles, permissions, uploads, Oracle transactions, responsive layouts and PWA behavior; then produce the final WAR and updated source ZIP.

Conversion rule for each screen

8. Identify the current React route and component.
9. Extract the HTML structure, styling, labels and expected behavior.
10. Create the matching JSP view under WEB-INF/views.
11. Create or update a Servlet controller for GET and POST requests.
12. Create DAO methods using prepared statements.
13. Add server-side validation and authorization.
14. Add only the minimum vanilla JavaScript required for interaction.
15. Test on mobile and desktop before moving to the next screen.

8. PWA, Security and File-Handling Rules

PWA requirements

- Provide manifest.json with application name, short name, start URL, standalone display mode, theme colour and 192x192 / 512x512 icons.
- Register service-worker.js from a context-path-safe JavaScript file.
- Cache only static assets such as CSS, JavaScript, icons and offline.html.
- Use an offline fallback page when the network is unavailable.
- Do not attempt offline approval, rejection, routing, user management or confidential document viewing in version 1.
- Production deployment must use HTTPS for service worker and camera-related browser features.

Security requirements

Area	Required implementation
Authentication	Use HttpSession after successful login. Regenerate the session identifier and invalidate the session during logout.
Authorization	Protect role-based URLs with filters and repeat authorization checks inside sensitive controllers.
Passwords	Store salted password hashes; never store or log plaintext passwords.
Database	Use PreparedStatement for every user-controlled value. Keep credentials outside source control.
CSRF	Use CSRF tokens for state-changing forms such as approve, reject, delete, recall and user management.
Output	Escape user-controlled text in JSP and avoid raw HTML injection.
Sessions	Use HttpOnly and Secure cookies in production and configure an appropriate session timeout.
Audit	Record user ID, action, date/time, document ID, old value, new value and remarks for important actions.

Document upload and storage rules

- Accept only approved types: PDF, JPG, JPEG, PNG, DOC, DOCX, XLS and XLSX.
- Validate file extension, MIME type and file signature where practical.
- Set clear per-file and per-request size limits.
- Generate a server-side storage name instead of trusting the uploaded filename.
- Prevent path traversal by resolving files only inside the configured storage directory.
- Store document metadata and the generated storage path in Oracle. Prefer filesystem or managed object storage for file bytes instead of placing all large files directly in database BLOB columns.
- Keep uploaded files outside the publicly accessible webapp directory and serve them through an authorized download servlet.
- Do not cache document downloads in the service worker.

Camera capture

For mobile capture, use an HTML file input with `accept="image/*"` and `capture="environment"`. Always retain normal file selection because browser camera behavior differs between devices.

9. Oracle Database Guidance

The exact schema may be adapted to existing database scripts, but the following entities should exist or be represented. The code owner should inspect and reuse any valid existing tables before creating replacements.

Suggested entity	Purpose
APP_USER	User account, name, username, password hash, role, department, status and timestamps.
ROLE	CLERK, BOSS, DEPARTMENT_USER and optional SYSTEM_ADMIN role definitions.
DEPARTMENT	IT, FINANCE, MANAGEMENT and future destination folders.
DOCUMENT	Document reference, title, description, original name, storage name/path, MIME type, size, owner, boss, destination, status, priority and dates.
APPROVAL	Document, approver, decision, remarks, previous/new destination and decision date.
DOCUMENT_HISTORY	Immutable audit trail of submission, edit, rejection, approval, routing, recall, archive and download actions.
NOTIFICATION	Recipient, message, type, read status, related document and timestamps.

Database implementation rules

- Use Oracle sequences or identity columns consistently for primary keys.

- Define foreign keys and useful indexes for status, boss, destination, submitter and creation date.
- Use transactions when a business action updates multiple tables, such as approval plus history plus notification.
- Use a connection pool for deployed environments rather than creating uncontrolled connections per request.
- Close ResultSet, PreparedStatement and Connection objects reliably by using try-with-resources.
- Do not hard-code the remote Oracle computer address or credentials in Java source files.

10. Acceptance Criteria

The migration is complete only when all applicable checks below pass.

- [] The project opens and builds with Maven without requiring the React/Vite frontend build.
- [] A WAR file is produced and deploys successfully on Apache Tomcat 9.
- [] The application connects to Oracle Database using configuration outside committed source code.
- [] Login, logout, session timeout and role-based access function correctly.
- [] Clerk, boss and department users see only the pages and records allowed for their roles.
- [] Document capture/upload validates supported file types and size limits.
- [] The clerk can select the boss and destination and submit a document.
- [] The boss can approve, reject and change destination with remarks.
- [] Rejected documents return to the clerk and may be corrected and resubmitted.
- [] Approved documents appear only in the correct department folder.
- [] Bulk selection and approval work if this feature exists in the current project scope.
- [] Document history records every material workflow action.
- [] The application is responsive on common phone and desktop widths.
- [] manifest.json, icons, service worker registration and installability are valid.
- [] Static resources can load from cache, but authenticated pages and confidential files are not cached.
- [] No React, JSX, TypeScript or Vite dependency remains unless explicitly documented as a temporary migration artifact.
- [] All user-visible text is checked for spelling, clipping, overflow and out-of-container layout errors.
- [] The final handover includes source code, SQL scripts, setup instructions, sample configuration and the deployable WAR file.

Required output from the modifying chat/developer

- A summary of what was found in each original ZIP file.
- A migration report listing reused, converted and removed files.
- The updated Maven source project in a downloadable ZIP.
- Oracle SQL scripts required to create or update the schema.
- A README with Java, Maven, Tomcat and Oracle setup steps.
- The final WAR file, when the environment supports building it.
- A list of any incomplete functions, assumptions or test limitations.

11. Ready-to-Paste Instruction for Another Chat

Copy the instruction below into the chat that will receive the current React/Vite ZIP files. Attach this PDF and both project ZIP files in the same conversation.

I am attaching the current Document Management System project ZIP files and a PDF named "Document Management System - React/Vite to Java JSP/Servlet PWA Migration Brief".

Please inspect every attached ZIP before changing the code. Identify what each project contains, how the frontend and backend currently communicate, which screens and features already work, and which files can be reused.

Then migrate the application according to the attached PDF. The required target is a mobile-first Java MVC Progressive Web Application using:

- Java JSP
- Java Servlets with javax.servlet for Tomcat 9
- JavaBeans / POJO models
- DAO + JDBC
- HTML, CSS and vanilla JavaScript
- Oracle Database 21c
- Apache Tomcat 9
- Maven WAR packaging
- manifest.json, service-worker.js and offline.html

Do not keep React, Vite, JSX or TypeScript as the final application architecture. Reuse suitable HTML structure, CSS, icons, images, labels, validation rules and business logic by translating them into JSP, Servlets, JavaBeans, DAO classes and vanilla JavaScript.

Preserve the required workflow: Clerk submits a captured or uploaded document to a selected Boss; the Boss approves, rejects or changes the destination; rejected documents return to the Clerk; approved documents are routed to IT, Finance or Management; department users can view only their own folder; important actions are recorded in history.

Important requirements:

1. Use Maven and produce one deployable WAR for Tomcat 9.
2. Use the javax.servlet namespace, not jakarta.servlet.
3. Protect pages using HttpSession and role-based filters.
4. Use PreparedStatement and secure password hashing.
5. Store uploaded files securely outside the public webapp directory and store metadata/path in Oracle.
6. Do not cache confidential documents or authenticated pages in the service worker.
7. Keep the approved colour scheme: blue #0297F1 for headers, green #ADCD20 as an accent, yellow #FDF11F where appropriate, white #FDFDFD and black #060004.
8. Make all pages mobile-first and also usable on desktop.
9. Check spelling, overflow, clipped text, broken links and layout problems after conversion.

Work incrementally. First provide an inventory and migration plan based on the actual attached files. Then perform the conversion and return the updated source as a downloadable ZIP, together with SQL scripts, README setup instructions, a migration summary and a list of anything not completed. Do not invent completed functionality; clearly state any limitations.

Final reminder

The target is not a native Android application. It is a responsive Java web application that can be installed as a PWA. Capacitor packaging may be considered later only if an APK is genuinely required.